

A Step-by-Step Source-Backed Derivation of DeepSeek-Style Multi-head Latent Attention

Codex long-horizon MLA run

2026-05-09

1 Evidence Boundary

This document formalizes the Multi-head Latent Attention (MLA) algebra used by the local DeepSeek-V2 profile under `math/mla/deepseekv2-profile`. The source hierarchy is recorded in `math/mla/source_ledger.md`; exact vLLM/SGLang comparison pins are recorded in `math/mla/external_sources.md`; shape notation is recorded in `math/mla/notation.md`.

The executable target for the core derivation is the simplified local algebra: no attention mask, dropout disabled, and base RoPE. The full model contains masking, past-cache mutation, dropout, and optional YaRN scaling. Those full-model details are important runtime features, but the equality proofs below concern the shared algebra after the same query tensors, compressed KV latent, and rotated key RoPE suffix have been produced.

2 Derivation Roadmap

The derivation proceeds in increasing specificity. First, ordinary attention is fixed as a template. Next, MHA, MQA, and GQA are used only to explain cache accounting. Then MLA is derived in ten steps:

1. define the low-rank projection maps;
2. derive the query path and split it into non-RoPE and RoPE parts;
3. derive the KV path and the compressed cache $[z, \rho]$;
4. materialize ordinary attention from the MLA tensors;
5. absorb the non-RoPE key projection into the query side;
6. keep RoPE separate because it is position dependent;
7. check that pinned external implementations use the same algebra;
8. state the executable shape assertions;
9. work a two-head numerical example by hand;
10. record the validation results and open limits.

Every equality promoted as an algebraic identity is either proved directly in finite-dimensional notation or tied to an executable parity check.

3 Dimensions

Symbol	Meaning	DeepSeek-V2 profile value
B	batch size	runtime
L	query sequence length	runtime
M	memory / key-value sequence length	runtime
D	model hidden dimension	5120
H	number of query heads	128
N	non-RoPE query/key head dimension	128
R	RoPE query/key suffix dimension	64
V	materialized value head dimension	128
Q	query compression rank	1536
C	KV compression rank	512
T	full query/key head width, $T = N + R$	192
X	packed KV down-projection width, $X = C + R$	576
Y	packed KV up-projection width per head, $Y = N + V$	256

Verification artifacts use shape suffixes built from these letters; in the derivation itself, shapes are written explicitly. For an integer n , write $[n] = \{1, \dots, n\}$.

4 Orientation and Algebraic Assumptions

All token vectors below are treated as row vectors. A linear map

$$P : \mathbb{R}^a \rightarrow \mathbb{R}^b$$

acts on the last dimension and produces row vectors in $\mathbb{R}^b$. This convention matches the tensor layout used in the shape assertions while keeping the mathematical maps explicit. In particular, for each head h ,

$$W_h^K \in \mathbb{R}^{N \times C}, \quad W_h^V \in \mathbb{R}^{V \times C},$$

so a latent row vector $z_m \in \mathbb{R}^{1 \times C}$ materializes as

$$k_{h,m}^n = z_m(W_h^K)^\top \in \mathbb{R}^{1 \times N}, \quad v_{h,m} = z_m(W_h^V)^\top \in \mathbb{R}^{1 \times V}.$$

The proof below uses the following assumptions.

1. The equations are for one attention layer; layer indices are omitted.
2. Dropout is disabled and all layer normalizations are evaluated identically in the materialized and absorbed paths.
3. The key and value up-projection matrices W_h^K, W_h^V are static and position independent. The base formulas treat these maps as linear.
4. RoPE is applied identically in both paths. Base RoPE is assumed in the executable reference checks; variants such as YaRN are listed as limits unless separately tested.
5. The attention scale is inherited from the materialized query/key width: $1/\sqrt{T} = 1/\sqrt{N+R}$. The absorbed path attends to a $(C+R)$ -wide cache object, but the scale is not changed to $1/\sqrt{C+R}$.

6. The simplified verifier uses no mask. If a shared additive mask $\mu_{b,\ell,m}$ is added to both forms of the logits, equality of logits before masking implies equality after masking.
7. Symbolic equalities are real-arithmetic equalities. Executable checks are floating-point parity checks and are therefore judged with explicit tolerances.

5 Attention Template

Before specializing to MLA, fix the ordinary attention calculation that all variants reuse. For a batch element b and head h , suppose

$$Q_{bh} \in \mathbb{R}^{L \times T}, \quad K_{bh} \in \mathbb{R}^{M \times T}, \quad V_{bh} \in \mathbb{R}^{M \times V}.$$

The score, attention, and context matrices are

$$\begin{aligned} S_{bh} &= \frac{Q_{bh} K_{bh}^\top}{\sqrt{T}} \in \mathbb{R}^{L \times M}, \\ A_{bh} &= \text{softmax}_{\text{row}}(S_{bh}) \in \mathbb{R}^{L \times M}, \\ O_{bh} &= A_{bh} V_{bh} \in \mathbb{R}^{L \times V}. \end{aligned}$$

After all heads are computed, the per-head contexts are concatenated to

$$O_b \in \mathbb{R}^{L \times HV}$$

and projected back to the model dimension by an output map

$$P_o : \mathbb{R}^{HV} \rightarrow \mathbb{R}^D.$$

This template separates two questions. First, how are Q, K, V produced? Second, what must be cached during autoregressive generation? MHA, MQA, GQA, and MLA differ mainly in the answer to those two questions.

6 Cache Ladder

For one layer, define the scalar cache count per memory token per batch element as

$$e(\mathcal{C}) = \frac{\text{elts}(\mathcal{C})}{BM}.$$

This count ignores dtype bytes and page padding; bytes are obtained by multiplying by the dtype size and any backend-specific storage overhead.

6.1 MHA

Standard multi-head attention stores separate key and value streams for every query head:

$$K^{\text{mha}} \in \mathbb{R}^{B \times H \times M \times T}, \quad V^{\text{mha}} \in \mathbb{R}^{B \times H \times M \times V}.$$

Therefore

$$\begin{aligned} e(\mathcal{C}_{\text{mha}}) &= \frac{BHMT + BHMV}{BM} \\ &= H(T + V). \end{aligned}$$

For the DeepSeek-V2 dimensions this is

$$128(192 + 128) = 40960$$

scalars per token per layer.

6.2 MQA

Multi-query attention keeps H query heads but shares one KV stream:

$$K^{\text{mqa}} \in \mathbb{R}^{B \times 1 \times M \times T}, \quad V^{\text{mqa}} \in \mathbb{R}^{B \times 1 \times M \times V}.$$

The cache count is

$$e(\mathcal{C}_{\text{mqa}}) = T + V.$$

For the profile values this is $192 + 128 = 320$.

6.3 GQA

Grouped-query attention has G KV groups serving H query heads. If G divides H , write

$$g(h) = 1 + \left\lfloor \frac{h-1}{H/G} \right\rfloor.$$

The cache is

$$K^{\text{gqa}} \in \mathbb{R}^{B \times G \times M \times T}, \quad V^{\text{gqa}} \in \mathbb{R}^{B \times G \times M \times V},$$

so

$$e(\mathcal{C}_{\text{gqa}}) = G(T + V).$$

MHA is the endpoint $G = H$; MQA is the endpoint $G = 1$.

6.4 MLA

If MLA materialized and cached all per-head keys and values, its cache would again be

$$H(T + V) = H(N + R + V).$$

Because the MLA RoPE suffix is shared across heads, a useful intermediate comparison is a semi-materialized cache that stores head-specific non-RoPE keys and values but stores the shared RoPE suffix once:

$$H(N + V) + R.$$

For the DeepSeek-V2 profile this semi-materialized count is

$$128(128 + 128) + 64 = 32832.$$

The compressed MLA cache instead stores only a normalized KV latent and a shared key RoPE suffix:

$$\mathcal{C}_{\text{mla}} = [z, \rho] \in \mathbb{R}^{B \times M \times (C+R)}.$$

Thus

$$e(\mathcal{C}_{\text{mla}}) = C + R.$$

For the DeepSeek-V2 profile this is

$$512 + 64 = 576$$

scalars per token per layer. This is larger than MQA’s scalar count in the same dimensions, but it retains head-specific key and value up-projections through W_h^K and W_h^V instead of forcing a single materialized KV head.

Design	Cache contents	Scalars per token
MHA	per-head K, V	$H(T + V) = 40960$
MQA	shared K, V	$T + V = 320$
GQA	G grouped K, V	$G(T + V)$
Materialized MLA	per-head constructed K, V	$H(T + V) = 40960$
Semi-materialized MLA	per-head non-RoPE K, V plus shared ρ	$H(N + V) + R = 32832$
Compressed MLA	latent z plus suffix ρ	$C + R = 576$

This table is the first key point: MLA is not just MQA with different dimensions. It stores one shared latent object, but that latent is later expanded through head-specific maps, so the resulting materialized keys and values can still be head dependent.

7 Step 1: Low-Rank Projection Maps

Let

$$h^q \in \mathbb{R}^{B \times L \times D}, \quad h^{kv} \in \mathbb{R}^{B \times M \times D}.$$

The learned maps are

$$\begin{aligned} P_{q,a} : \mathbb{R}^D &\rightarrow \mathbb{R}^Q, & P_{q,b} : \mathbb{R}^Q &\rightarrow \mathbb{R}^{HT}, \\ P_{kv,a} : \mathbb{R}^D &\rightarrow \mathbb{R}^{C+R}, & P_{kv,b} : \mathbb{R}^C &\rightarrow \mathbb{R}^{H(N+V)}, & P_o : \mathbb{R}^{HV} &\rightarrow \mathbb{R}^D. \end{aligned}$$

The layer normalizations act on the last dimension:

$$\text{LN}_q : \mathbb{R}^Q \rightarrow \mathbb{R}^Q, \quad \text{LN}_{kv} : \mathbb{R}^C \rightarrow \mathbb{R}^C.$$

The source ledger maps these abstract projections to local implementation modules. The derivation below uses only their mathematical domains and codomains.

8 Step 2: Query Path

The local query path is

$$\begin{aligned} q^{\text{lat}} &= \text{LN}_q(P_{q,a}(h^q)) \in \mathbb{R}^{B \times L \times Q}, \\ \bar{q} &= P_{q,b}(q^{\text{lat}}) \in \mathbb{R}^{B \times L \times HT}. \end{aligned}$$

Viewing the packed last dimension as heads gives

$$q_{b,h,\ell,t}^{\text{full}} = \bar{q}_{b,\ell,(h-1)T+t} \in \mathbb{R}^{B \times H \times L \times T}.$$

Split

$$q^{\text{full}} = [q^{\text{n}}, q^{\text{r,raw}}],$$

where

$$q^{\text{n}} \in \mathbb{R}^{B \times H \times L \times N}, \quad q^{\text{r,raw}} \in \mathbb{R}^{B \times H \times L \times R}.$$

For query position id p_ℓ^q , RoPE acts only on the suffix:

$$q_{b,h,\ell,:}^{\text{r}} = \text{RoPE}_{p_\ell^q}(q_{b,h,\ell,:}^{\text{r,raw}}) \in \mathbb{R}^R.$$

The full query used by materialized attention is

$$Q_{b,h,\ell,:} = [q_{b,h,\ell,:}^{\text{n}}, q_{b,h,\ell,:}^{\text{r}}] \in \mathbb{R}^T.$$

The query RoPE suffix is therefore produced by the query low-rank path and then rotated. It is not produced by the KV RoPE projection.

9 Step 3: KV Path and Compressed Cache

The packed KV down-projection is

$$u = P_{kv,a}(h^{kv}) \in \mathbb{R}^{B \times M \times (C+R)}.$$

Split it as

$$u = [c, k^{r,raw}], \quad c \in \mathbb{R}^{B \times M \times C}, \quad k^{r,raw} \in \mathbb{R}^{B \times M \times R}.$$

Normalize the latent and rotate the RoPE suffix:

$$z = \text{LN}_{kv}(c) \in \mathbb{R}^{B \times M \times C},$$

$$\rho_{b,m,:} = \text{RoPE}_{p_m^{kv}}(k_{b,m,:}^{r,raw}) \in \mathbb{R}^R.$$

The compressed cache token is

$$\mathcal{C}_{b,m,:} = [z_{b,m,:}, \rho_{b,m,:}] \in \mathbb{R}^{C+R}.$$

The packed KV up-projection weight can be viewed per head as

$$W^{KV} \in \mathbb{R}^{H \times (N+V) \times C}.$$

For a latent row vector z_m , write

$$\bar{w}_{h,j} = \sum_{c=1}^C W_{h,j,c}^{KV} z_{m,c}, \quad h \in [H], j \in [N+V].$$

Split

$$W_h^K \in \mathbb{R}^{N \times C}, \quad W_h^V \in \mathbb{R}^{V \times C}.$$

Materialized non-RoPE keys and values are

$$k_{b,h,m,n}^n = \sum_{c=1}^C W_{h,n,c}^K z_{b,m,c} \in \mathbb{R}^{B \times H \times M \times N},$$

$$v_{b,h,m,\nu} = \sum_{c=1}^C W_{h,\nu,c}^V z_{b,m,c} \in \mathbb{R}^{B \times H \times M \times V}.$$

The full materialized key is

$$K_{b,h,m,:} = [k_{b,h,m,:}^n, \rho_{b,m,:}] \in \mathbb{R}^T,$$

where ρ is broadcast across heads.

10 Step 4: Materialized MLA Forward Pass

For $b \in [B]$, $h \in [H]$, query position $\ell \in [L]$, and memory position $m \in [M]$, materialized MLA first constructs ordinary attention vectors

$$Q_{b,h,\ell,:} = [q_{b,h,\ell,:}^n, q_{b,h,\ell,:}^r] \in \mathbb{R}^T,$$

$$K_{b,h,m,:} = [k_{b,h,m,:}^n, \rho_{b,m,:}] \in \mathbb{R}^T,$$

and

$$V_{b,h,m,:} = v_{b,h,m,:} \in \mathbb{R}^V.$$

The ordinary unnormalized attention score is

$$r_{b,h,\ell,m} = \langle Q_{b,h,\ell,:}, K_{b,h,m,:} \rangle.$$

Because Q and K are concatenations of non-RoPE and RoPE coordinates, this score decomposes as

$$r_{b,h,\ell,m} = \langle q_{b,h,\ell,:}^n, k_{b,h,m,:}^n \rangle + \langle q_{b,h,\ell,:}^r, \rho_{b,m,:} \rangle.$$

The scaled score is

$$s_{b,h,\ell,m} = \frac{r_{b,h,\ell,m}}{\sqrt{T}}.$$

The simplified local verifier uses no mask. If a shared additive mask $\mu_{b,\ell,m}$ is present, it is added to the same logits in both forms below and does not change the algebraic equivalence.

Attention probabilities are

$$a_{b,h,\ell,m} = \frac{\exp(s_{b,h,\ell,m})}{\sum_{j=1}^M \exp(s_{b,h,\ell,j})}.$$

The materialized context and output projection are

$$o_{b,h,\ell,\nu} = \sum_{m=1}^M a_{b,h,\ell,m} v_{b,h,m,\nu} \in \mathbb{R}^{B \times H \times L \times V},$$

$$y_{b,\ell,:} = P_o(\text{flatten}_H(o_{b,:,\ell,:})) \in \mathbb{R}^D.$$

11 Step 5: Projection Absorption

11.1 Key-Side Absorption

The non-RoPE score can be evaluated without materializing k^n . Define the absorbed query latent

$$q'_{b,h,\ell,c} = \sum_{n=1}^N q_{b,h,\ell,n}^n W_{h,n,c}^K = (q_{b,h,\ell,:}^n W_h^K)_c \in \mathbb{R}^C.$$

Then

$$\begin{aligned} \langle q_{b,h,\ell,:}^n, k_{b,h,m,:}^n \rangle &= \sum_{n=1}^N q_{b,h,\ell,n}^n \sum_{c=1}^C W_{h,n,c}^K z_{b,m,c} \\ &= \sum_{c=1}^C \left(\sum_{n=1}^N q_{b,h,\ell,n}^n W_{h,n,c}^K \right) z_{b,m,c} \\ &= \langle q'_{b,h,\ell,:}, z_{b,m,:} \rangle. \end{aligned}$$

Thus the absorbed score is

$$r'_{b,h,\ell,m} = \langle q'_{b,h,\ell,:}, z_{b,m,:} \rangle + \langle q_{b,h,\ell,:}^r, \rho_{b,m,:} \rangle,$$

and $r' = r$ exactly in real arithmetic.

11.2 Delayed Value Up-Projection

After softmax, compute a latent context

$$g_{b,h,\ell,c} = \sum_{m=1}^M a_{b,h,\ell,m} z_{b,m,c} \in \mathbb{R}^{B \times H \times L \times C}.$$

The value-side up-projection can then be applied after the memory sum:

$$\begin{aligned} \sum_{c=1}^C W_{h,\nu,c}^V g_{b,h,\ell,c} &= \sum_{c=1}^C W_{h,\nu,c}^V \sum_{m=1}^M a_{b,h,\ell,m} z_{b,m,c} \\ &= \sum_{m=1}^M a_{b,h,\ell,m} \sum_{c=1}^C W_{h,\nu,c}^V z_{b,m,c} \\ &= \sum_{m=1}^M a_{b,h,\ell,m} v_{b,h,m,\nu} = o_{b,h,\ell,\nu}. \end{aligned}$$

The absorbed and materialized forms therefore produce the same pre-output context, and hence the same final P_o output, up to floating-point evaluation order.

12 Step 6: Why RoPE Is Separated

The absorption step uses a static weight W_h^K and an unrotated latent $z_{b,m,:}$:

$$q^n \left(z_m (W_h^K)^\top \right)^\top = q^n W_h^K z_m^\top = (q^n W_h^K) z_m^\top.$$

This rearrangement is possible because the non-RoPE key map is linear and position independent. The RoPE term is position dependent:

$$\left\langle \text{RoPE}_{p_\ell^q}(q_{b,h,\ell,:}^{\text{r},\text{raw}}), \text{RoPE}_{p_m^{kv}}(k_{b,m,:}^{\text{r},\text{raw}}) \right\rangle.$$

It is therefore kept as an explicit R -dimensional suffix $\rho_{b,m,:}$ in the cache. The compressed cache is not merely $z \in \mathbb{R}^C$; it is $[z, \rho] \in \mathbb{R}^{C+R}$.

A sharper way to see the obstruction is to write RoPE as a position-dependent orthogonal block rotation

$$\text{RoPE}_p(x) = \mathcal{R}_p x, \quad \mathcal{R}_p \in \mathbb{R}^{R \times R}.$$

For query position p^q and key position p^{kv} ,

$$\langle \mathcal{R}_{p^q} x, \mathcal{R}_{p^{kv}} y \rangle = x^\top \mathcal{R}_{p^q}^\top \mathcal{R}_{p^{kv}} y.$$

The middle matrix depends on the relative position. A single static projection could absorb the RoPE term for all query/key positions only if

$$A = \mathcal{R}_{p^q}^\top \mathcal{R}_{p^{kv}}$$

were the same matrix for all (p^q, p^{kv}) , which is false for non-degenerate RoPE. The usual relative-position identity is useful, but it is not a static head weight that can be folded into W_h^K .

13 Step 7: External Implementation Check

Source-backed facts. The pinned vLLM snapshot independently describes the same two algorithms: the compute-friendly path materializes

$$k^n = z(W^K)^\top, \quad v = z(W^V)^\top,$$

then runs ordinary multi-head attention; the data-movement-friendly path forms $q' = q^n W^K$, attends against the latent cache z , and applies W^V after attention. Its MLA cache spec uses one KV head and semantic cache width $C + R$.

The pinned SGLang snapshot implements the same absorbed decode shape through an MQA-style attention object with head dimension $C + R$, one KV head, and value head dimension C . Its memory pool allocates one MLA KV buffer per layer and writes the latent part followed by the RoPE suffix into that single buffer. The default symbolic cache width is $C + R$, with separate FP8/NSA storage exceptions.

Runtime shape contract. A runtime vLLM probe was also executed with a one-layer DeepSeek-V2 config and dummy weights, so no official weights were downloaded. With the profile dimensions, the observed attention inputs had shapes

$$\mathbb{R}^{S \times H \times T} = \mathbb{R}^{S \times 128 \times 192}, \quad \mathbb{R}^{S \times C} = \mathbb{R}^{S \times 512}, \quad \mathbb{R}^{S \times 1 \times R} = \mathbb{R}^{S \times 1 \times 64}.$$

The recorded module dimensions were

$$\begin{aligned} H &= 128, & C &= 512, & N &= 128, \\ R &= 64, & X &= C + R = 576. \end{aligned}$$

The generated pre-output tensor had width $128 \cdot 128 = 16384 = HV$, consistent with concatenating all materialized value heads before the output projection.

Mathematical contract. For each fixed (b, h, ℓ) , let

$$\alpha_m = \text{softmax}_m \left(\frac{q^n W_h^K z_m^\top + q^r \rho_m^\top}{\sqrt{T}} \right).$$

The compute-friendly output is

$$o_\nu^{\text{mat}} = \sum_{m=1}^M \alpha_m \sum_{c=1}^C W_{h,\nu,c}^V z_{m,c}.$$

The data-movement-friendly output is

$$o_\nu^{\text{abs}} = \sum_{c=1}^C W_{h,\nu,c}^V \sum_{m=1}^M \alpha_m z_{m,c}.$$

By exchanging the finite sums,

$$o_\nu^{\text{mat}} = o_\nu^{\text{abs}}.$$

The logits defining α_m are also identical because

$$q^n W_h^K z_m^\top = (q^n W_h^K) z_m^\top.$$

This is exactly the algebraic object both external implementations optimize. Paged-cache tables, quantized cache layouts, sparse/NSA indexers, CUDA graph constraints, and specialized kernels change storage or execution strategy, not the base equalities above.

14 Step 8: Executable Shape Assertions

The verification scripts assert the following mathematical shape evolution. The query path is

$$\begin{aligned} h^q \in \mathbb{R}^{B \times L \times D} &\longmapsto q^{\text{lat}} \in \mathbb{R}^{B \times L \times Q} \longmapsto q^{\text{full}} \in \mathbb{R}^{B \times H \times L \times T}, \\ q^{\text{full}} &\longmapsto q^n \in \mathbb{R}^{B \times H \times L \times N}, \quad q^r \in \mathbb{R}^{B \times H \times L \times R}. \end{aligned}$$

The KV path is

$$\begin{aligned} h^{kv} \in \mathbb{R}^{B \times M \times D} &\longmapsto u \in \mathbb{R}^{B \times M \times (C+R)} \longmapsto z \in \mathbb{R}^{B \times M \times C}, \quad \rho \in \mathbb{R}^{B \times M \times R}, \\ [z, \rho] &\in \mathbb{R}^{B \times M \times (C+R)}. \end{aligned}$$

The materialized path asserts

$$\begin{aligned} z &\longmapsto k^n \in \mathbb{R}^{B \times H \times M \times N}, \quad v \in \mathbb{R}^{B \times H \times M \times V}, \\ [q^n, q^r] &\in \mathbb{R}^{B \times H \times L \times T}, \\ [k^n, \rho] &\in \mathbb{R}^{B \times H \times M \times T}, \\ s &\in \mathbb{R}^{B \times H \times L \times M}, \\ o &\in \mathbb{R}^{B \times H \times L \times V}, \quad y \in \mathbb{R}^{B \times L \times D}. \end{aligned}$$

The absorbed path asserts

$$\begin{aligned} q' &\in \mathbb{R}^{B \times H \times L \times C}, \\ [q', q^r] &\text{ attends against } [z, \rho] \in \mathbb{R}^{B \times M \times (C+R)}, \\ g &\in \mathbb{R}^{B \times H \times L \times C}, \\ g(W_h^V)^\top &\in \mathbb{R}^{B \times H \times L \times V}, \quad y \in \mathbb{R}^{B \times L \times D}. \end{aligned}$$

15 Step 9: A Two-Head Manual Forward Pass

This example is intentionally small enough to check by hand. Let

$$B = 1, \quad L = 1, \quad M = 2, \quad D = 2, \quad H = 2, \quad Q = C = 1, \quad N = V = 1, \quad R = 2.$$

Then $T = N + R = 3$. Choose zero RoPE suffixes so that the RoPE-shaped tensors are present but contribute zero to the score:

$$q^r = 0, \quad \rho_1 = \rho_2 = 0.$$

Let the compressed memory latents be

$$z_1 = 1, \quad z_2 = 2.$$

Let the two non-RoPE query scalars be

$$q_{h=1}^n = 2, \quad q_{h=2}^n = -1.$$

Choose per-head up-projection scalars

$$W_1^K = 1, \quad W_1^V = 2, \quad W_2^K = -1, \quad W_2^V = 1.$$

The materialized keys and values are

$$k_1^n = (1, 2), \quad v_1 = (2, 4),$$

$$k_2^n = (-1, -2), \quad v_2 = (1, 2).$$

The scale is $1/\sqrt{3}$. Head 1 scores are

$$s_1 = \left(\frac{2}{\sqrt{3}}, \frac{4}{\sqrt{3}} \right) \approx (1.1547, 2.3094),$$

so

$$a_1 = \text{softmax}(s_1) \approx (0.2396, 0.7604),$$

and

$$o_1 = 0.2396 \cdot 2 + 0.7604 \cdot 4 \approx 3.5208.$$

Head 2 scores are

$$s_2 = \left(\frac{1}{\sqrt{3}}, \frac{2}{\sqrt{3}} \right) \approx (0.5774, 1.1547),$$

so

$$a_2 = \text{softmax}(s_2) \approx (0.3595, 0.6405),$$

and

$$o_2 = 0.3595 \cdot 1 + 0.6405 \cdot 2 \approx 1.6405.$$

The absorbed form gives the same numbers without materializing the non-RoPE keys. Its absorbed query scalars are

$$q'_1 = q_1^n W_1^K = 2, \quad q'_2 = q_2^n W_2^K = 1.$$

Thus the logits are unchanged:

$$q'_1(z_1, z_2) = (2, 4), \quad q'_2(z_1, z_2) = (1, 2),$$

before applying the common scale $1/\sqrt{3}$. The latent contexts are

$$g_1 = 0.2396 \cdot 1 + 0.7604 \cdot 2 \approx 1.7604, \quad g_2 = 0.3595 \cdot 1 + 0.6405 \cdot 2 \approx 1.6405.$$

Applying the value-side weights after the sum gives

$$W_1^V g_1 \approx 2 \cdot 1.7604 = 3.5208, \quad W_2^V g_2 \approx 1 \cdot 1.6405 = 1.6405.$$

With identity output projection,

$$y \approx (3.5208, 1.6405) \in \mathbb{R}^{1 \times 1 \times 2}.$$

The script `math/mla/verification/two_head_manual_example.py` records the same calculation and outputs

$$[3.520736883716041, 1.6404574756806274].$$

16 Step 10: Verification Summary

The main verifier ran on an NVIDIA B200 GPU with `torch==2.11.0+cu128`. The deterministic check used

$$B = 2, L = 3, M = 5, D = 16, H = 4, N = 3, R = 2, V = 3, Q = 5, C = 6.$$

All checked equalities passed:

Check	Max absolute error
Independent reference vs local baseline	$5.96 \cdot 10^{-8}$
Concatenated score vs decomposed score	$1.19 \cdot 10^{-7}$
Concatenated score vs absorbed score	$1.19 \cdot 10^{-7}$
Materialized context vs absorbed context	$5.96 \cdot 10^{-8}$
Compressed-cache implementation vs baseline	0
Absorbed implementation vs baseline	$5.96 \cdot 10^{-8}$

The article export was also reproduced with `tools/zhihu-to-markdown/export_article.py`; LaTeX integrity passed with 101 fragments. The TeX build result is tracked in `math/mla/verification/tex_build.md`.

Two vLLM-oriented checks were added. The standalone contract trace verifies the algebra on both a small shape and the profile shape; the profile run has output max absolute difference $2.62 \cdot 10^{-6}$, consistent with large float32 reductions. The actual vLLM dummy-engine probe loads the one-layer DeepSeek-V2 config, captures the MLA tensor shapes above, and generates one token with dummy weights.

17 Open Limits

The current equality proof is for the shared dense MLA algebra. It extends immediately to any additive mask that is inserted identically into the materialized and absorbed logits before softmax.

The proof does not claim bitwise floating-point identity. It claims real-arithmetic equality plus executable floating-point parity within the tolerances recorded in the verification artifacts.

The following production concerns remain outside this base theorem: training-time dropout, YaRN-specific scaling changes, all mask variants not represented as a shared additive logit term, mutation semantics of a production paged cache, quantized cache formats, sparse/NSA attention patterns, CUDA graph constraints, and backend-specific fused kernels. The production implementations confirm the same base cache and absorption equations, but each of those runtime features should receive its own proof or parity test before it is folded into the mathematical statement.